

Principles of

**Scalable Performance in
Parallel Computing**

Parallelism Profile in Programs

- **Degree of Parallelism** For each time period, the number of processors used to execute a program is defined as the degree of parallelism (DOP).
- The plot of the DOP as a function of time is called the parallelism profile of a given program.
- Fluctuation of the profile during an observation period depends on the algorithmic structure, program optimization, resource utilization, and run-time conditions of a computer system.

Average Parallelism

- The *average parallelism* A is computed by:

$$A = \left(\sum_{i=1}^m i \cdot t_i \right) / \left(\sum_{i=1}^m t_i \right)$$

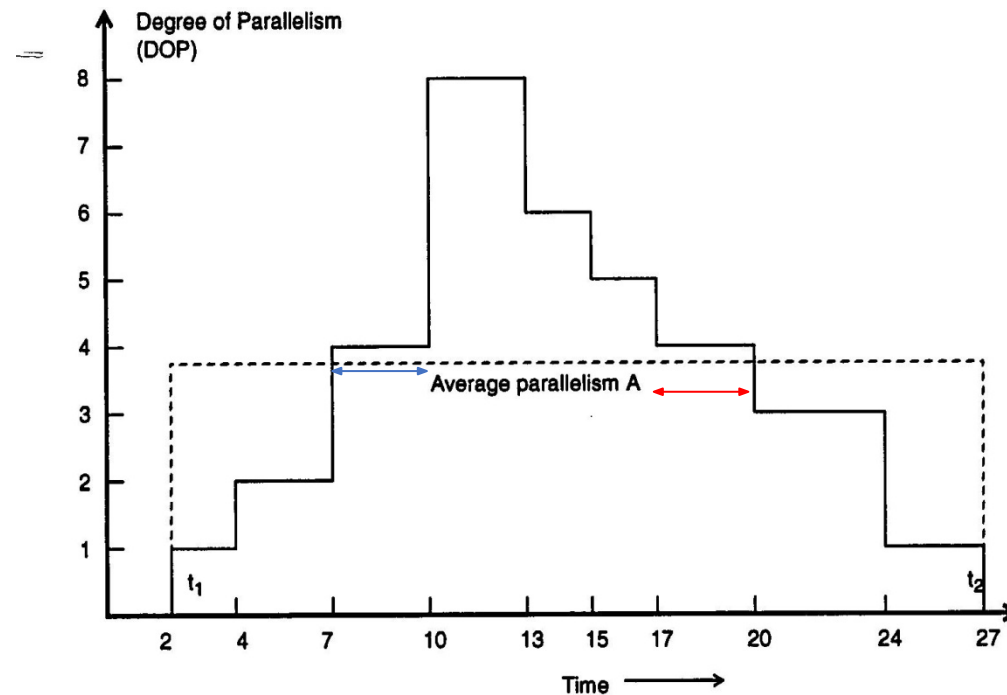
where:

- m is the maximum parallelism in a profile
- t_i is the total amount of time that DOP = i

$$\sum_{i=1}^m t_i = t_2 - t_1$$

- Is the total elapsed time
- i is the no. of processors

Example



- The parallelism profile of an example divide-and-conquer algorithm (Sun and Ni 1993) increases from 1 to its peak value $m = 8$ and then decreases to 0 during the observation period (t_1, t_2) .

$$A = (1 \times 5 + 2 \times 3 + 3 \times 4 + 4 \times 6 + 5 \times 2 + 6 \times 2 + 8 \times 3) / (5 + 3 + 4 + 6 + 2 + 2 + 3) = 93 / 25 = 3.72.$$

Note: blue line plus red line is t_i

Speedup

- Speedup is the ratio of the execution time of the best possible serial algorithm on a single processor $T(1)$ to the parallel execution time of the chosen algorithm on n-processor parallel system $T(n)$:

$$S(n) = T(1)/T(n)$$

- Speedup measure the absolute merits of parallel algorithms with respect to the “optimal” sequential version.

Efficiency

- The *system efficiency* for an n -processor system:

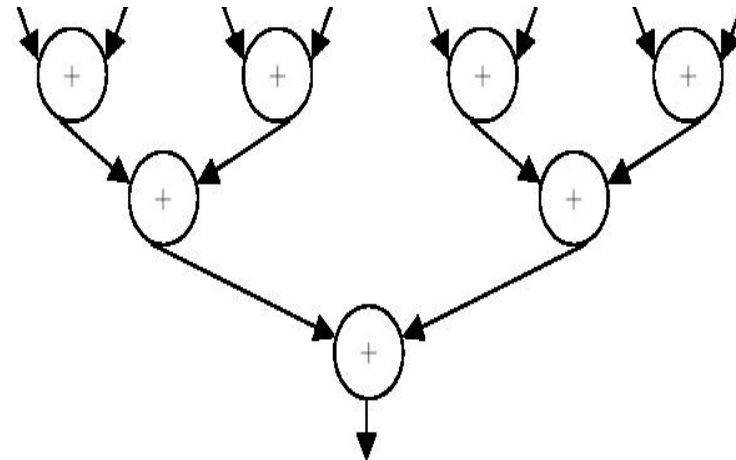
$$E(n) = \frac{S(n)}{n} = \frac{T(1)}{nT(n)}$$

- Efficiency is a measure of the speedup achieved per processor.

Scalability of Parallel Algorithms [1]

- Scalability analysis determines whether parallel processing of a given problem can offer the desired improvement in performance.
- Parallel system is scalable if its efficiency can be kept fixed as the number of processors is increased assuming that the problem size is also increased.
- *Example:* Adding m numbers of process using n processors. Communication and computation take one unit time.
- Steps:
 1. Each processor adds m/n numbers
 2. The processors combine their sums

$$S = \frac{m}{m/n + 2\log_2 n}$$



Mean Performance

- Arithmetic mean performance

$$R_a = \sum_{i=1}^m R_i / m$$

Where:

- m is the no. of programs executed by n number of processors.
- R_i is the execution rate of programs $i = 1, 2, 3, \dots, m$
- The execution rate R_i is measured in MIPS (million instructions per second) and MFLOPS (million floating point operations per second)

Amdahl's Law

The potential **speedup** of an algorithm on parallel computing platform is given by Amdahl's Law; originally formulated by Gene Amdahl's on 1960's

It is one of the **speedup performance law**

It is based on a **fixed problem size (or fixed work load)**

Actually *to keep the efficiency of system fixed* we have to increase both the size of the problem and the no. of processors simultaneously

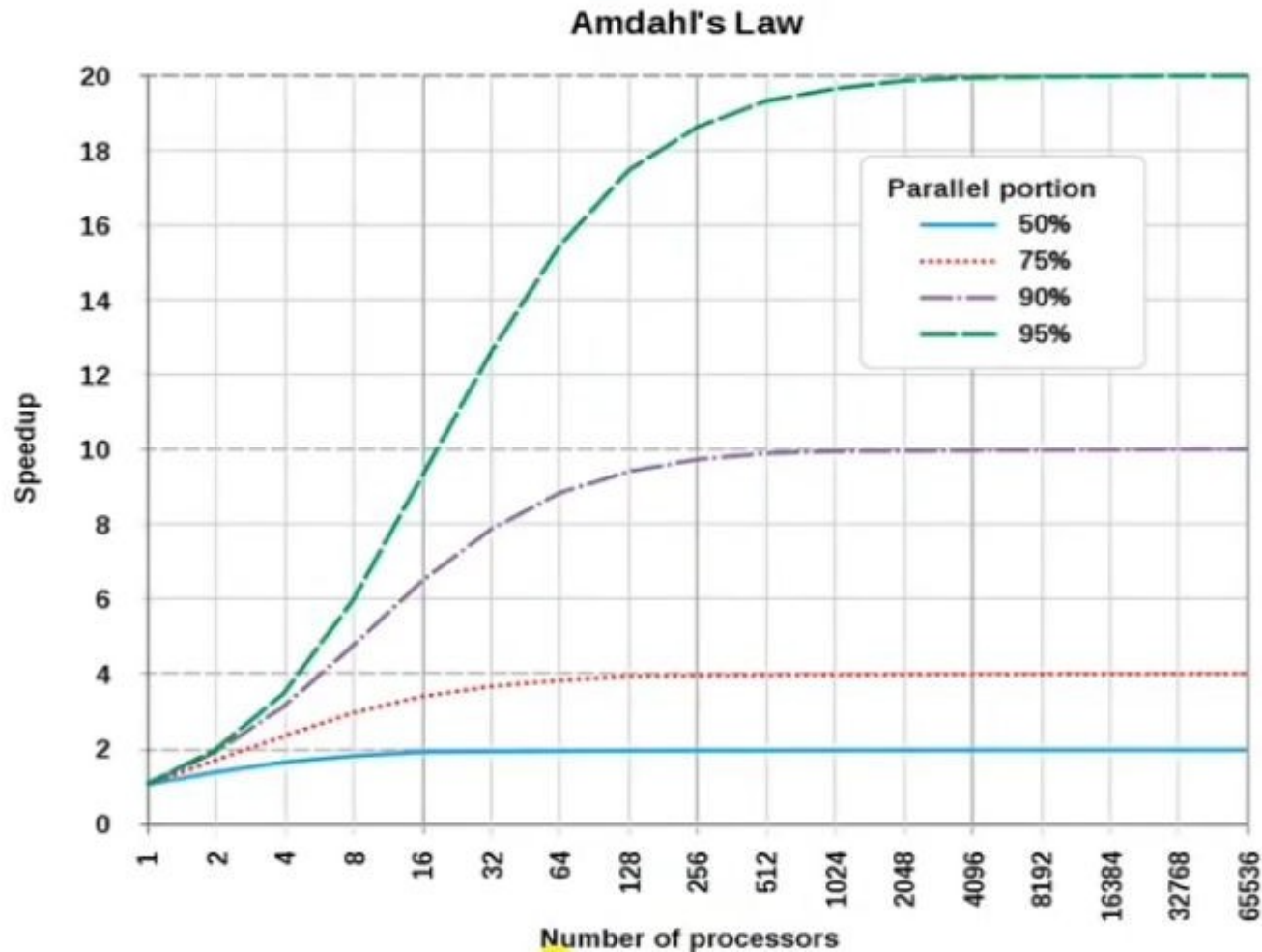
Amdahl's Law tells that for a given problem size, *the speedup doesn't increase linearly as the number of processor increases. In fact, speed-up tends to become saturated*

Amdahl's law states that a small portion of the program which cannot be parallelized (serial part), will limit the overall speed-up, available from parallelization

- ▣ Typically, any large mathematical or engineering problem consists of several parallelizable parts & several serial (non-parallelizable) parts

Cont...

The **speedup** of a program using multiple processors in parallel computing is **limited** by the time needed for the **sequential fraction (non-parallelizable part)** of the program.



Cont...

Let ' f ' be the fraction of operations in a computation that must be performed **sequentially**, where $0 \leq f \leq 1$. The **maximum speedup achievable by a parallel computer with n processors is**

Maximum Speedup

$$S(n) \leq \underbrace{f}_{\text{Serial Part}} + \underbrace{\frac{1-f}{n}}_{\text{Parallel Part}}$$

Serial Part **Parallel Part**

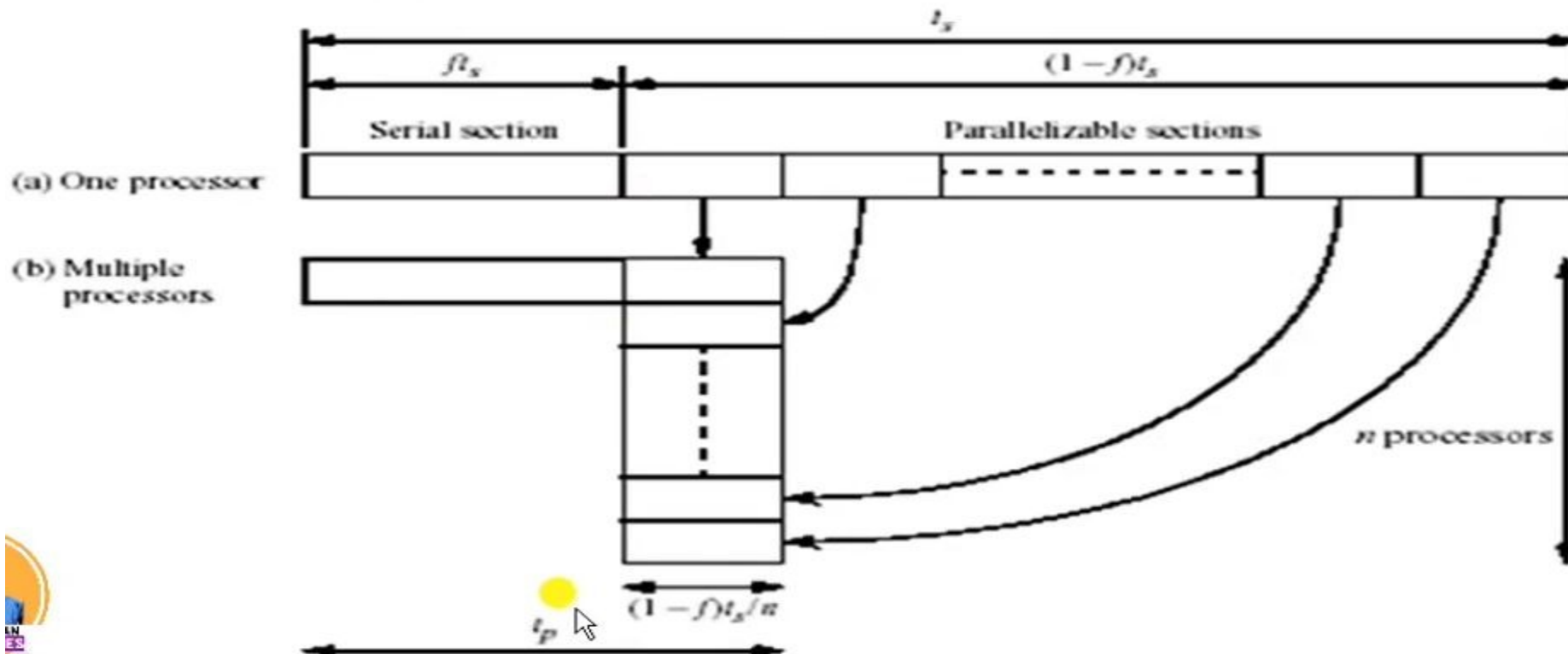
Proof of Amdahl's law

If the fraction of computation that can be executed in serial is '**f**' (**serial section**) and the remaining computation that can be divided into concurrent parts and executed in parallel is **(1-f)** i.e. **parallelizable section**, then the time to perform the computation with **single processor** (t_s) and multiple '**n**' processors (t_p) is given by:

$$t_s = ft_s + (1-f)t_s$$

$$= ft_s + t_s - ft_s \text{ (i.e. } = t_s \text{)}$$

$$t_p \geq ft_s + \frac{(1-f)t_s}{n}$$



In multiprocessor, with a *single serial part* at the beginning of computation, but the parallel part could be distributed throughout the computation to different processors

The overall speedup factor is given by

Hence, the **Overall Speedup factor** is given by :

$$S(n) = \frac{T(1)}{T(n)} = \frac{t_s}{t_p} = \frac{\text{Serial Execution time on single processor}}{\text{Parallel Execution time on } n \text{ multiple processor}}$$

$$S(n) = \frac{t_s}{t_p} \leq \frac{t_s}{ft_s + \frac{(1-f)t_s}{n}}$$
$$= \frac{1}{f + \frac{(1-f)}{n}}$$

$$S(n) \leq \underbrace{f}_{\text{Serial Part}} + \underbrace{\frac{(1-f)}{n}}_{\text{Parallel Part}}$$

Serial Part Parallel Part

This is known as Amdahl's Law

By further solving we get:

$$S(n) \leq \frac{n}{nf + (1-f)} = \frac{n}{1 + (n-1)f}$$

Numericals on Amdahl's law

70% of a program's execution time occurs inside a loop that can be executed in **parallel** and rest **30%** in **serial**. What is the maximum speedup we should expect from a parallel version of the program executing on 8 CPUs?

$$S(n) = \frac{1}{\frac{0.30 + (1 - 0.30)}{8}} = \frac{1}{0.3875} = 2.6$$

$$S(n) \leq \frac{1}{f + \frac{(1-f)}{n}}$$

Numericals on Amdahl's law

- ❑ GATE CS – Assume a microprocessor given which is widely used for scientific applications. It has both integer and floating point instructions. Now floating point instructions are enhanced and made 3 times faster than before and integer instructions are unenhanced. If there are 20% of floating point instructions in the program then find the overall speed up.

Shortcomings of Amdahl's Law

The work load cannot scale to match the available computing power as the machine size increases. Hence, the fixed load prevents scalability in performance

The sequential bottleneck is a serious problem

Amdahl's Law Example: $f = 0.95$

$$n = 5: \quad S(5) = 4.17$$

$$n = 10: \quad S(10) = 6.9$$

$$n = 100: \quad S(100) = 16.9$$

$$n \rightarrow \infty: \quad S(\infty) = 20$$

How many processors do you want to pay for?

77% of the most powerful supercomputers have $n > 4000$ processing units.

Is that many processor cores efficient?

What type of problems are these systems designed for?



Gustafson's Law (for scaled problems)

- One of the major drawback of Amdahl's law is that the problem size is fixed and it cannot scale to meet the available computing power as the machine size increases.
- Gustafson's law removes the fixed load size restriction.
- Amdahl's law:

$$S_n = \frac{n}{1 + (n-1)\alpha}$$

Here α is equivalent to f used in previous equations.

Where: alpha is the processor in sequential mode (let 30% in sequential i.e., 0.3 is alpha)

- **Gustafson's law;**

$$S'_n = \frac{\alpha + n(1-\alpha)}{\alpha + (1-\alpha)} = n - \alpha(n-1)$$